Threat Intelligence & IOC

Analisi di una cattura di rete e identificazione degli indicatori di compromissione

Nicola Madonna

7 agosto 2026

Riassunto e obiettivi

Questo report documenta l'analisi della cattura di rete `Cattura_U3_W1_L3.pcapng` fornita nell'esercizio pratico sulla Threat Intelligence e gli Indicatori di Compromissione (IOC). Come richiesto dalla traccia, gli obiettivi del lavoro sono tre:

1. identificare ed analizzare eventuali IOC, ovvero evidenze di un attacco in corso;
2. formulare un'ipotesi sul vettore di attacco utilizzato, sulla base degli IOC individuati;
3. proporre delle contromisure per ridurre l'impatto dell'attacco osservato e prevenirne di simili in futuro.

Oltre ai tre punti richiesti, il lavoro è stato approfondito con un ulteriore obiettivo non strettamente richiesto dalla traccia ma ritenuto utile a fini didattici: ricostruire, a partire dagli indicatori tecnici presenti nella cattura, il comando `nmap` più probabilmente utilizzato per generare il traffico osservato, e verificare l'ipotesi riproducendo lo scan in un ambiente di laboratorio isolato, confrontando la cattura ottenuta con quella originale.

1 Ambiente e strumenti utilizzati

L'analisi è stata condotta con **Wireshark** su una macchina virtuale **Kali Linux**, gestita tramite Oracle VirtualBox. Il file di cattura è stato trasferito sulla VM ed aperto direttamente in Wireshark per l'ispezione dei pacchetti.
Per la seconda fase di verifica sperimentale (Sezione 5) è stata inoltre allestita una seconda macchina virtuale, **Metasploitable2** (verificata tramite `uname -a`: kernel `2.6.24-16-server`, hostname `metasploitable`), collegata alla Kali tramite una *rete interna* VirtualBox isolata, non raggiungibile né da Internet né dalla rete fisica dell'host. Alle due macchine sono stati assegnati staticamente gli stessi indirizzi IP presenti nella cattura originale (`192.168.200.100` per Kali, `192.168.200.150` per Metasploitable2), per rendere il confronto più diretto.

2 Analisi della cattura

2.1 Panoramica del traffico

La cattura contiene complessivamente 2083 pacchetti, raccolti in un arco temporale di circa 37 secondi. Analizzando le conversazioni di rete (*Statistics* → *Conversations*, scheda IPv4) emerge immediatamente un dato significativo: praticamente la totalità del traffico (2078 pacchetti su 2083) è concentrata in un'unica conversazione tra due soli host, `192.168.200.100` e `192.168.200.150`, con un rapporto di pacchetti scambiati quasi 1:1 (1052 in una direzione, 1026 nell'altra).

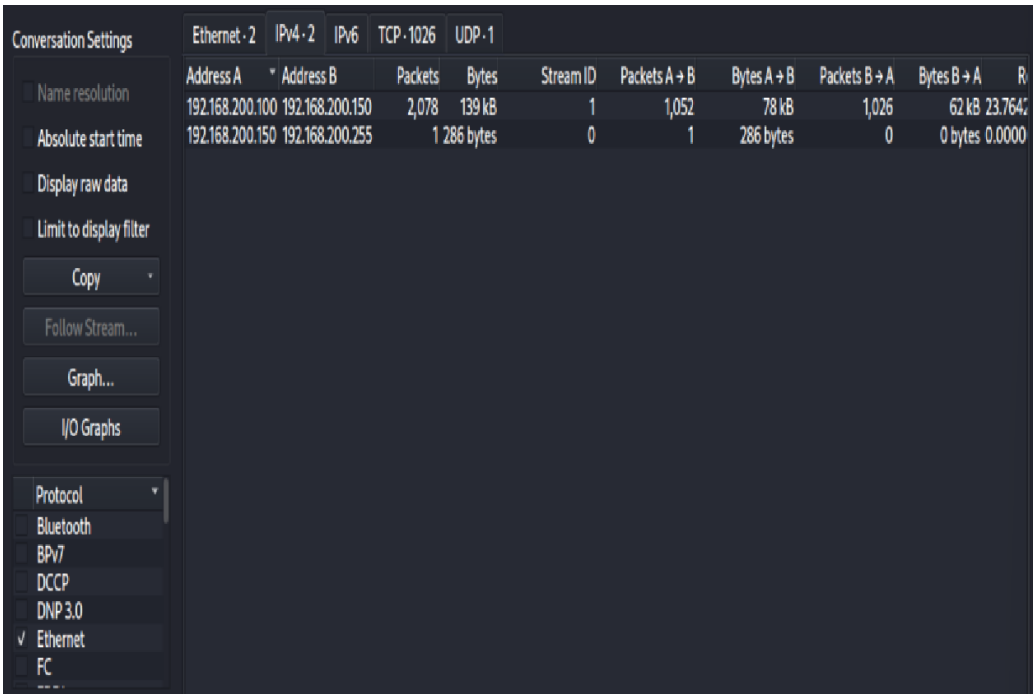


Figura 1: Conversazioni IPv4 della cattura originale: quasi tutto il traffico è concentrato tra i due host `192.168.200.100` e `192.168.200.150`.

L'unica altra conversazione rilevante è un singolo pacchetto broadcast UDP (porta 138, protocollo NetBIOS/SMB Browser) inviato da `192.168.200.150` verso l'indirizzo di broadcast `192.168.200.255`, in cui l'host annuncia se stesso con l'hostname **"METASPLOITABLE"** – un primo indizio importante sull'identità del bersaglio, approfondito nella Sezione 2.5.

2.2 Identificazione dell'IOC principale: port scan

Applicando il filtro di visualizzazione

```
tcp.flags.syn==1 && tcp.flags.ack==0
```

emergono **1026 pacchetti TCP SYN** inviati da `192.168.200.100` verso `192.168.200.150`. Analizzando questi pacchetti si osservano le seguenti caratteristiche:

- le porte di destinazione coprono l'intero intervallo **da 1 a 1024**, ciascuna contattata esattamente una volta (con l'eccezione delle porte 80 e 443, toccate due volte a causa di un probe isolato precedente);
- l'ordine con cui le porte vengono contattate **non è sequenziale**, ma appare casuale;
- la porta sorgente cambia ad ogni pacchetto in modo apparentemente casuale;
- per le porte chiuse (la stragrande maggioranza) il target risponde direttamente con un RST, senza alcun handshake, come atteso;
- per le **porte aperte**, invece, l'analisi puntuale della sequenza di pacchetti (Sezione 3) mostra che il client **completa sempre il three-way handshake** inviando l'ACK finale dopo il SYN,ACK ricevuto dal target, per poi chiudere immediatamente la connessione con un RST,ACK.

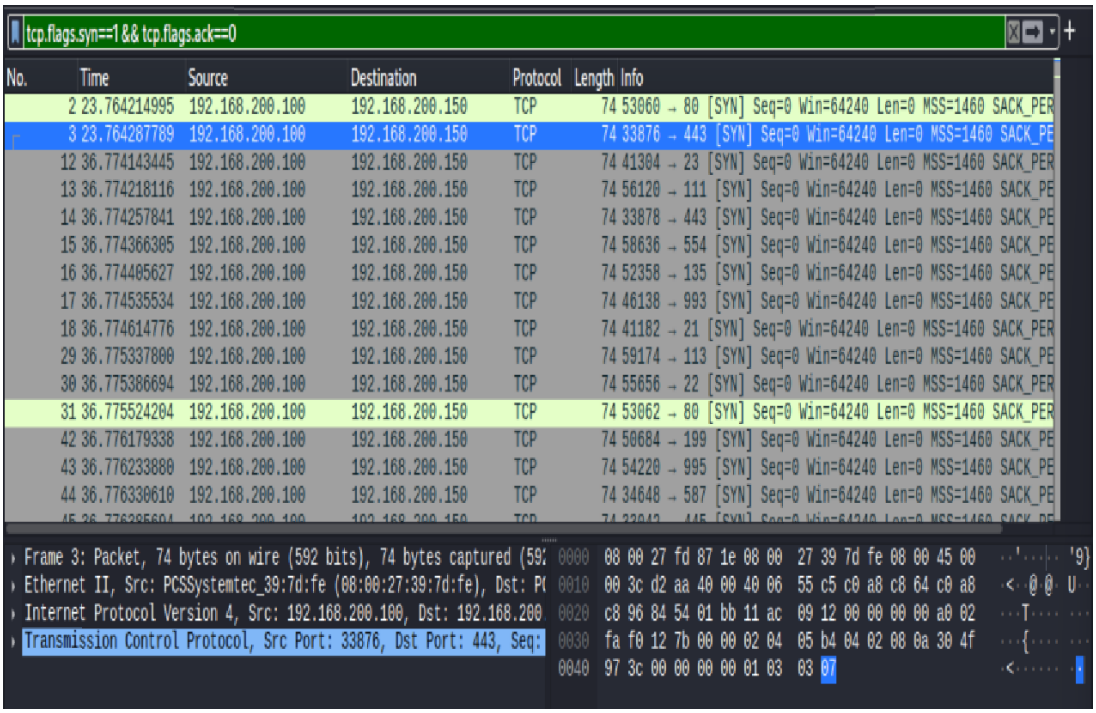


Figura 2: Pacchetti SYN inviati da `192.168.200.100` verso `192.168.200.150`: si notano porte di destinazione non sequenziali.

Questo comportamento, ovvero quello di avere moltissime richieste di connessione verso porte diverse, in rapidissima successione, ciascuna chiusa immediatamente dopo aver determinato lo stato della porta – è la firma tipica di una **scansione di porte TCP** (*port scan*), tecnica di *network reconnaissance* classificata in MITRE ATT&CK come **T1046 – Network Service Scanning**. Il tipo esatto di scan (SYN stealth vs. TCP Connect) viene determinato nella Sezione 3 sulla base del comportamento osservato per le porte aperte.
Un secondo elemento rafforza fortemente questa ipotesi: analizzando i timestamp dei pacchetti, l'intera scansione (i 1024 SYN dell'intervallo 1–1024) si svolge in una finestra temporale di soli **∼105 millisecondi** (dal secondo 36.774 al secondo 36.879 della cattura), corrispondente a un tasso di invio superiore ai **9500 pacchetti al secondo**. Una velocità di questo tipo è incompatibile con un utilizzo manuale o con normale traffico applicativo, ed è invece coerente con uno strumento di scansione automatizzato configurato con un profilo di timing aggressivo.

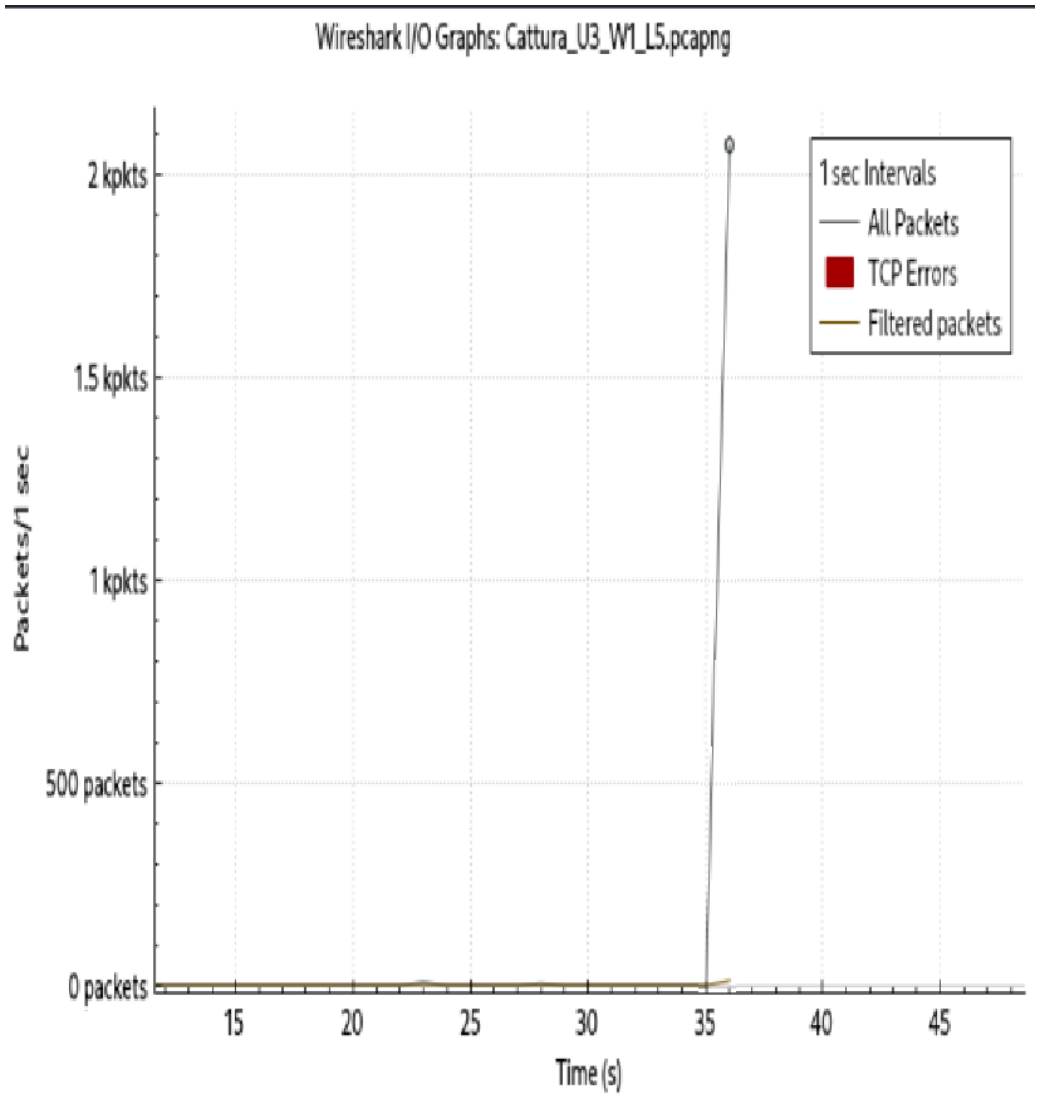


Figura 3: Grafico I/O della cattura originale: il picco isolato di oltre 2000 pacchetti/secondo intorno al secondo 36 corrisponde alla scansione di porte.

2.3 Fingerprint tecnico dei pacchetti di scansione

Tutti i 1026 pacchetti SYN condividono esattamente le stesse caratteristiche a livello IP e TCP, riportate in Tabella 1. L'uniformità totale di questi valori su oltre mille pacchetti conferma che il traffico è generato da un unico strumento automatico, e non da connessioni applicative reali.

Tabella 1: Caratteristiche costanti dei pacchetti SYN della scansione

Campo	**Valore**
TTL	64
Window size	64240
Opzioni TCP	MSS=1460, SACK_PERM, Timestamps, NOP, Window Scale=7 (128)
Porta sorgente	variabile / apparentemente casuale
Porta destinazione	range 1–1024, ordine non sequenziale

2.4 Porte aperte individuate

Applicando il filtro

```
tcp.flags.syn==1 && tcp.flags.ack==1
```

si isolano le risposte del target che confermano una porta aperta. La scansione ha rivelato **12 porte aperte** sull'host `192.168.200.150`, elencate in Tabella 2.

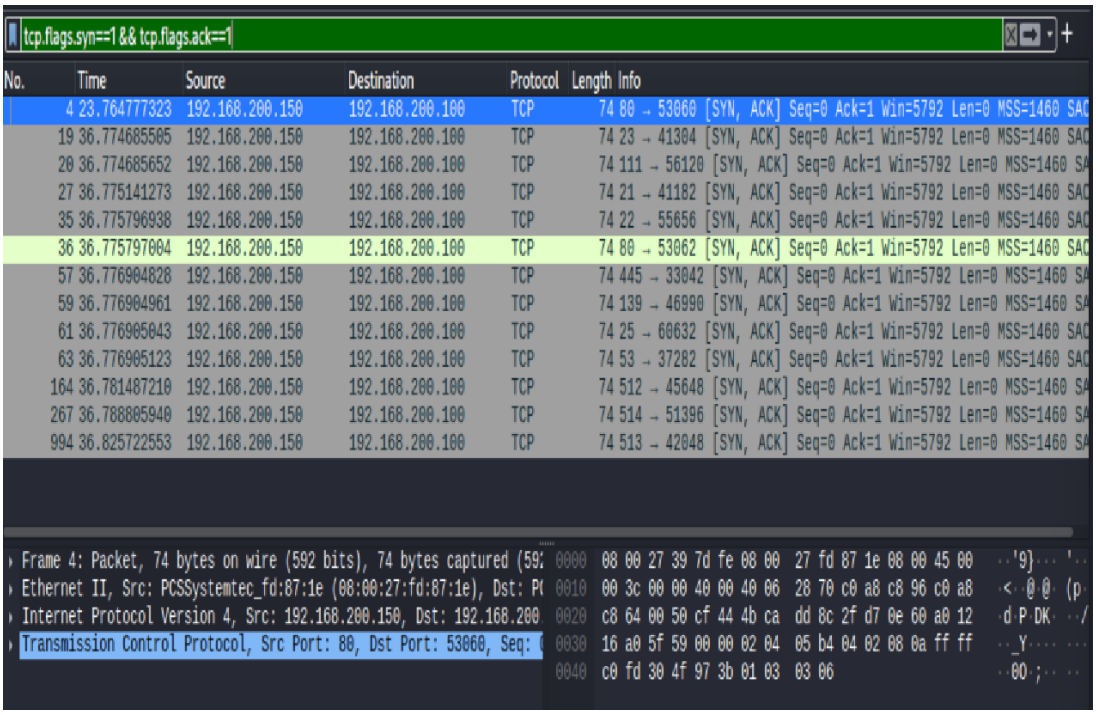


Figura 4: Risposte SYN,ACK del target nella cattura originale: le porte effettivamente aperte.

Tabella 2: Porte aperte individuate sul target

Porta	Servizio
21	FTP
22	SSH
23	Telnet
25	SMTP
53	DNS
80	HTTP
111	rpcbind
139	NetBIOS-SSN
445	Microsoft-DS / SMB
512	exec (rexec)
513	login (rlogin)
514	shell (rsh)

2.5 Identificazione del bersaglio

Diversi elementi, presi insieme, permettono di identificare con buona sicurezza il sistema operativo e la natura della macchina bersaglio, anche senza accesso diretto ad essa (*OS fingerprinting passivo*):

- il pacchetto broadcast iniziale annuncia l'hostname **"metasploitable"**;
- il TTL delle risposte è **64**, tipico delle famiglie Linux/Unix (Windows di default parte da 128);
- sono presenti i cosiddetti *r-services* Unix (porte 512, 513, 514), mai implementati nativamente su Windows;
- è aperta la porta 111 (`rpcbind`, il portmapper Unix), mentre la porta 135 (MSRPC, l'equivalente Windows) risulta chiusa nonostante sia stata sondata dalla scansione.

L'insieme di questi elementi è coerente con **Metasploitable2**, la macchina virtuale Linux (basata su Ubuntu 8.04).

3 Ipotesi sul vettore di attacco

3.1 Determinazione del tipo di scansione

Analizzando nel dettaglio la sequenza completa di pacchetti scambiati per ciascuna delle 13 porte che hanno risposto con SYN,ACK (le 12 porte aperte trovate dallo scan più il probe isolato iniziale sulla porta 80), emerge un pattern costante e determinante:

```
SYN (client) -> SYN,ACK (target) -> ACK (client) -> RST,ACK (client)
```

In **tutti e 13 i casi**, senza eccezioni, il client completa il three-way handshake con un ACK esplicito, per poi chiudere la connessione quasi immediatamente con un RST,ACK proveniente dal client stesso. Questo comportamento, unito al fatto che i pacchetti SYN in uscita presentano le opzioni TCP tipiche di un vero stack di rete Linux (MSS, SACK_PERM, Timestamps, NOP, Window Scale, con una window size di 64240 – si veda la Tabella 1), è incompatibile con una scansione SYN "half-open" (`-sS`), che tipicamente *non* completa l'handshake e utilizza pacchetti grezzi con un template di opzioni proprio di Nmap, diverso da quello del kernel.
È invece pienamente coerente con una **scansione TCP Connect (`-sT`)**: questa modalità utilizza le normali system call di rete del sistema operativo (`connect()`) anziché socket raw, per cui ogni probe passa realmente attraverso lo stack TCP/IP del kernel – da qui la presenza delle opzioni TCP "genuine" e il completamento dell'handshake per le porte aperte, seguito da una chiusura immediata della connessione una volta determinato lo stato della porta. Va inoltre notato che `-sT` è la modalità di scansione che Nmap adotta automaticamente quando eseguito senza privilegi elevati (senza accesso a socket raw), il che rende plausibile anche questa spiegazione per l'assenza di uno scan `-sS` nella cattura originale.

3.2 Ricostruzione del comando

Sulla base di tutti gli indicatori raccolti, il traffico osservato è compatibile con una scansione:

```
nmap -sT -p 1-1024 <target>
```

con un profilo di timing molto aggressivo (compatibile con `-T4`/`-T5` o un `-min-rate` esplicito, viste le velocità superiori ai 9500 pacchetti al secondo osservate durante il picco della scansione). La sequenza temporale della cattura supporta ulteriormente questa ipotesi:

1. un primo probe isolato verso le porte 80 e 443 (verosimilmente una verifica manuale di raggiungibilità del servizio web, precedente all'attacco automatizzato);
2. una risoluzione ARP dell'indirizzo del target, tipico comportamento di *host discovery* di Nmap su una rete Ethernet locale;

3. la scansione vera e propria, con le caratteristiche descritte in Sezione 2.2.

Si tratta quindi, con ogni probabilità, della fase di **ricognizione** (network reconnaissance) che tipicamente precede lo sfruttamento di uno dei servizi vulnerabili individuati – Metasploitable2 è notoriamente affetta, tra le altre, da vulnerabilità critiche su vsftpd (porta 21), Samba (porte 139/445) e sui servizi r-* (porte 512–514), tutte scoperte come aperte in questa scansione.

4 Contromisure consigliate

4.1 Per ridurre l'impatto dell'attacco osservato

- Rilevare e allertare in tempo reale le attività di port scanning tramite un IDS/IPS di rete (es. Suricata/Snort con preprocessore di *portscan detection*), basato su soglie tipo "N porte diverse contattate dallo stesso IP in una finestra di X secondi".
- Restringere via firewall/ACL l'accesso all'host esposto solo agli indirizzi realmente autorizzati, isolandolo dal resto della rete.
- Applicare rate-limiting sulle nuove connessioni in ingresso (es. modulo `recent` di `iptables`, o strumenti come fail2ban) per rallentare scansioni aggressive.

4.2 Per prevenire attacchi futuri simili

- Disabilitare o rimuovere i servizi legacy non necessari e intrinsecamente insicuri individuati (Telnet, FTP, i servizi r-* su 512–514), sostituendoli con alternative cifrate (SSH al posto di Telnet e rlogin, SFTP al posto di FTP).
- Segmentare la rete separando gli host che espongono servizi critici dal resto dell'infrastruttura.
- Mantenere aggiornati i servizi esposti e monitorare periodicamente le porte effettivamente in ascolto sui sistemi, per individuare servizi non necessari o dimenticati.
- Valutare l'impiego di honeypot o strumenti di *deception* per ottenere un allarme precoce in caso di attività di ricognizione sulla rete.

5 Verifica sperimentale in laboratorio

Per validare l'ipotesi formulata in Sezione 3, lo scan è stato riprodotto in un ambiente di laboratorio isolato composto dalla Kali e dalla Metasploitable2 descritte in Sezione 2.5, collegate esclusivamente tramite rete interna VirtualBox e configurate con gli stessi indirizzi IP della cattura originale.

5.1 Primo tentativo: SYN scan

Un primo tentativo di riproduzione è stato eseguito con:

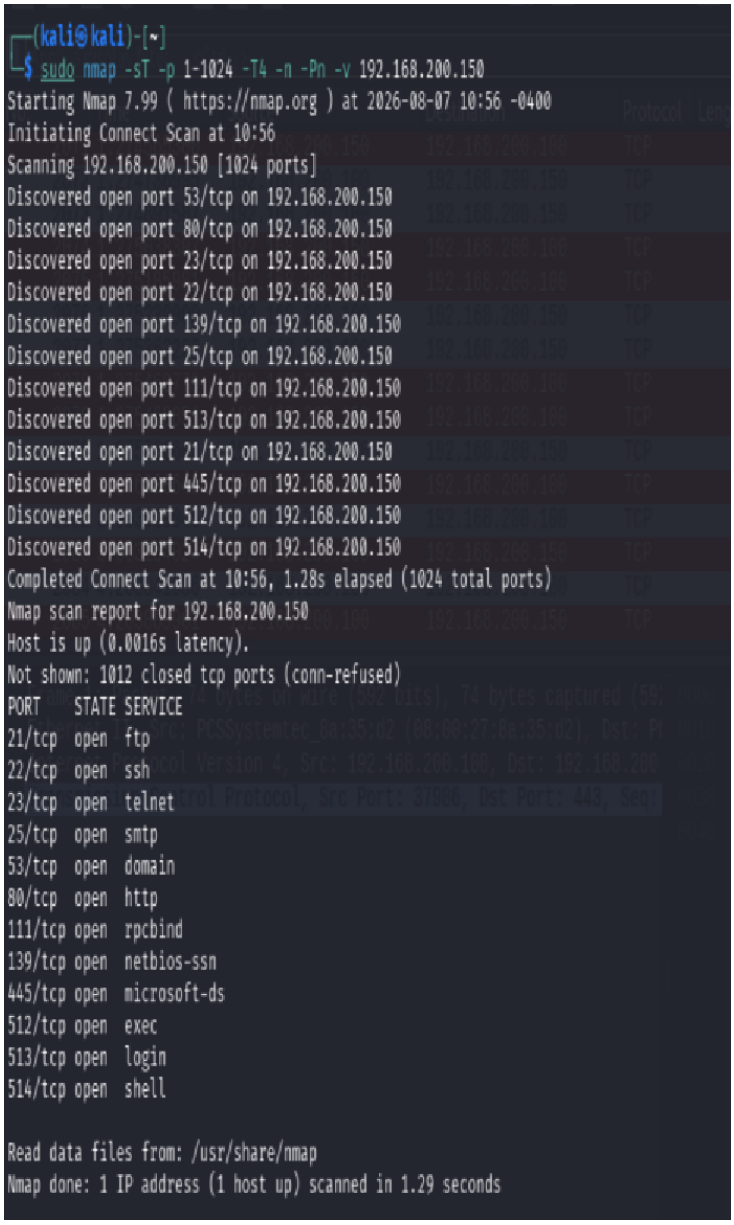
```
sudo nmap -sS -p 1-1024 -T4 -n -Pn -v 192.168.200.150
```

Pur trovando le stesse 12 porte aperte, la cattura ottenuta presentava differenze significative rispetto all'originale: *window size* pari a 1024 (invece di 64240), solo l'opzione TCP `MSS` presente (senza SACK, Timestamps o Window Scale) e nessun pacchetto di ACK a completamento dell'handshake sulle porte aperte. Queste discrepanze sono state proprio l'indizio che ha portato a formulare l'ipotesi `-sT` descritta in Sezione 3.

5.2 Secondo tentativo: TCP Connect scan

Il comando è stato quindi corretto in:

```
sudo nmap -sT -p 1-1024 -T4 -n -Pn -v 192.168.200.150
```



```
┌──(kali㉿kali)-[~]
└─$ sudo nmap -sT -p 1-1024 -T4 -n -Pn -v 192.168.200.150
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-07 10:56 -0400
Initiating Connect Scan at 10:56
Scanning 192.168.200.150 [1024 ports]
Discovered open port 53/tcp on 192.168.200.150
Discovered open port 80/tcp on 192.168.200.150
Discovered open port 23/tcp on 192.168.200.150
Discovered open port 22/tcp on 192.168.200.150
Discovered open port 139/tcp on 192.168.200.150
Discovered open port 25/tcp on 192.168.200.150
Discovered open port 111/tcp on 192.168.200.150
Discovered open port 513/tcp on 192.168.200.150
Discovered open port 21/tcp on 192.168.200.150
Discovered open port 445/tcp on 192.168.200.150
Discovered open port 512/tcp on 192.168.200.150
Discovered open port 514/tcp on 192.168.200.150
Completed Connect Scan at 10:56, 1.28s elapsed (1024 total ports)
Nmap scan report for 192.168.200.150
Host is up (0.0016s latency).
Not shown: 1012 closed tcp ports (conn-refused)
PORT    STATE SERVICE
21/tcp  open  ftp
22/tcp  open  ssh
23/tcp  open  telnet
25/tcp  open  smtp
53/tcp  open  domain
80/tcp  open  http
111/tcp open  rpcbind
139/tcp open  netbios-ssn
445/tcp open  microsoft-ds
512/tcp open  exec
513/tcp open  login
514/tcp open  shell

Read data files from: /usr/share/nmap
Nmap done: 1 IP address (1 host up) scanned in 1.29 seconds
```

Figura 5: Output del comando `nmap -sT` eseguito in laboratorio.

Il risultato conferma pienamente l'ipotesi:

- le **stesse identiche 12 porte aperte** vengono individuate;
- la sequenza `SYN` → `SYN,ACK` → `ACK` viene completata, come nella cattura originale (*Initiating Connect Scan* nel log di Nmap, non più *SYN Stealth Scan*);
- le opzioni TCP tornano ad essere quelle "genuine" del kernel: window size 64240, con MSS, SACK_PERM e Timestamps;

- il numero di pacchetti totali scambiati (2083) è praticamente identico a quello della cattura originale (2078).

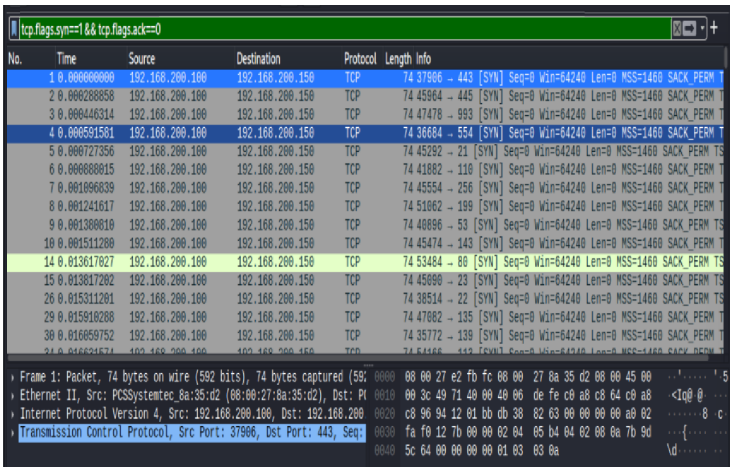


Figura 6: Pacchetti SYN della scansione riprodotta in laboratorio.

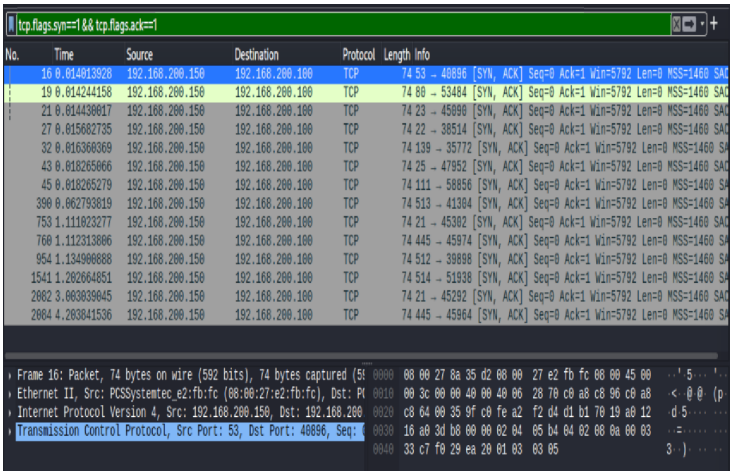


Figura 7: Risposte SYN,ACK della scansione riprodotta: le stesse 12 porte aperte dell'originale.

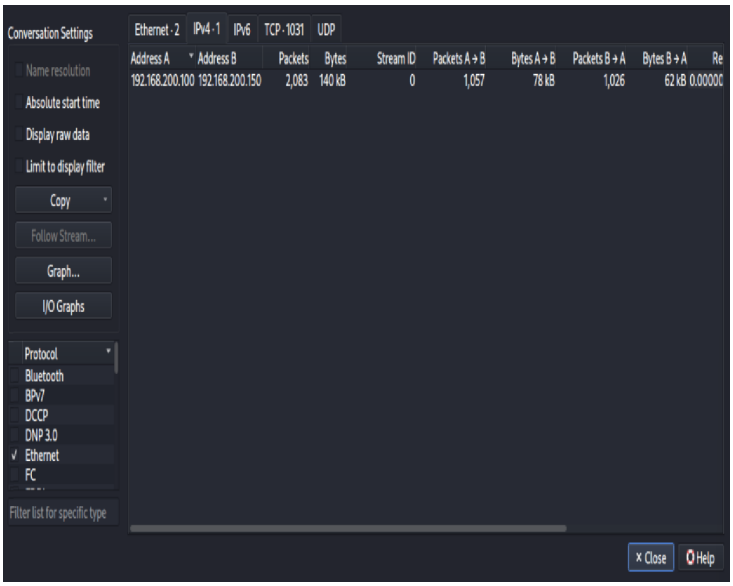


Figura 8: Conversazioni IPv4 della cattura riprodotta in laboratorio.

L'unica differenza degna di nota è la **durata complessiva** della scansione: circa 1.29 secondi nell'ambiente di laboratorio, contro i ∼105 millisecondi della cattura originale. Come atteso per una scansione `-sT`, ogni probe richiede l'apertura di una connessione reale a livello di kernel, con un overhead strutturalmente maggiore rispetto a una scansione `-sS` basata su pacchetti raw: lo stesso confronto interno (0.35s per lo scan `-sS` di prova contro 1.29s per quello `-sT`, sulla stessa identica macchina) conferma questo rapporto. La differenza rispetto ai ∼105ms dell'originale è quindi attribuita a fattori ambientali – probabile minore overhead di virtualizzazione della rete sulla macchina su cui è stata generata la cattura originale – e non contraddice l'ipotesi sul tipo di scansione utilizzata, che risulta pienamente confermata.

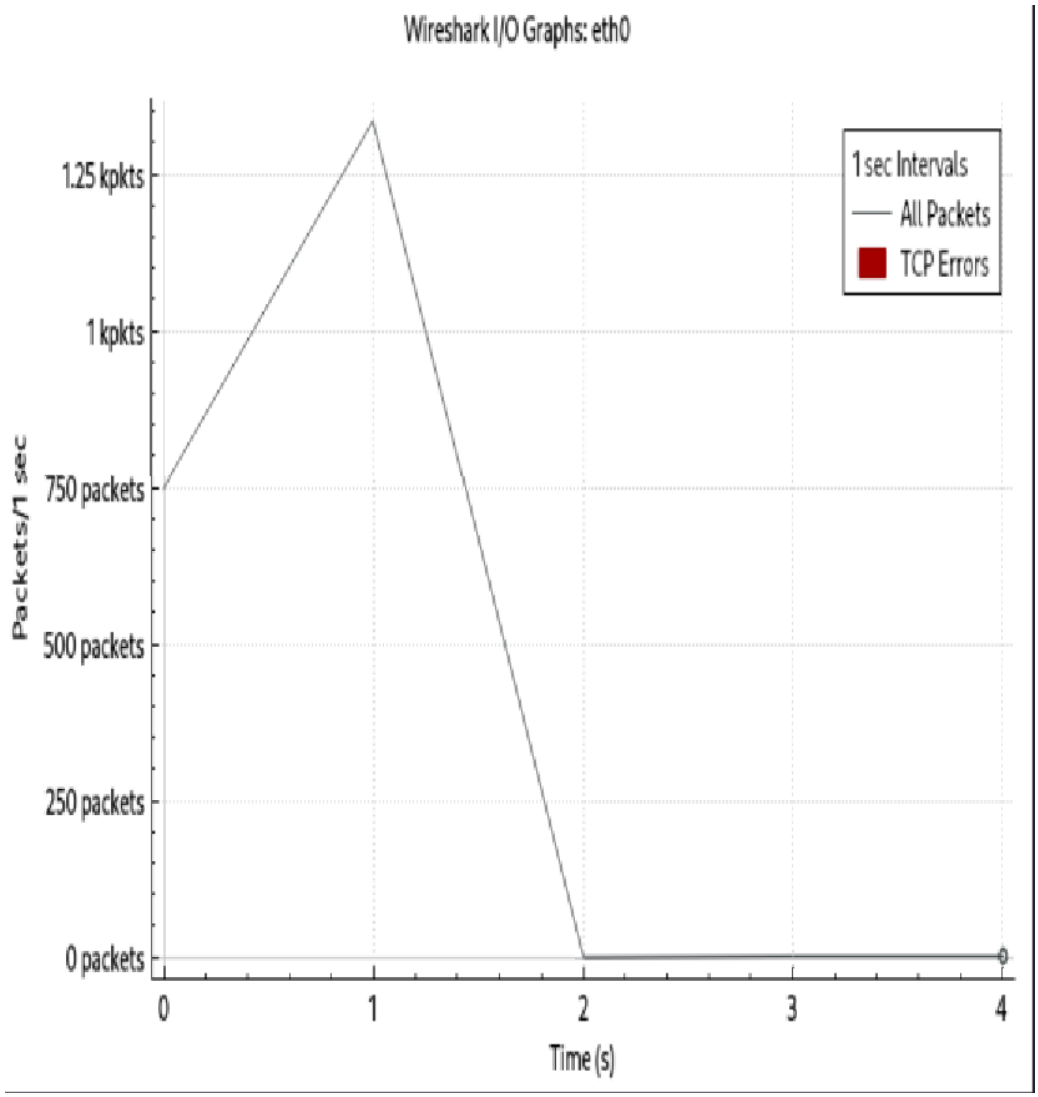


Figura 9: Grafico I/O della scansione riprodotta: il picco di traffico, più esteso nel tempo rispetto all'originale ma con la stessa firma qualitativa.

5.3 Conclusioni della verifica

La riproduzione in laboratorio conferma che il traffico della cattura originale è generato da una scansione **Nmap TCP Connect (`-sT`)** sul range di porte `1-1024`, con timing aggressivo. L'identità del bersaglio (Metasploitable2) e l'elenco delle porte vulnerabili esposte sono anch'essi confermati dalla riproduzione, rafforzando le conclusioni riportate nelle Sezioni 3 e 2.5.

6 Conclusioni

Il lavoro svolto ha permesso di rispondere in modo completo e verificato ai tre quesiti posti dalla traccia, seguendo un percorso metodologico che ricalca da vicino il flusso di lavoro reale di un analista di Threat Intelligence: dalla semplice osservazione di un pattern anomalo di traffico, fino alla validazione sperimentale dell'ipotesi formulata.
A partire da un unico indicatore superficiale, ovvero da un numero elevato di connessioni verso porte diverse in pochissimo tempo – l'analisi è stata approfondita fino a ricostruire con precisione lo strumento (`Nmap`), la tecnica specifica (`-sT`, TCP Connect scan) e il comando utilizzato, sfruttando indicatori a più livelli: il pattern temporale della scansione, il fingerprint delle opzioni

TCP e, soprattutto, il comportamento esatto dell'handshake sulle porte aperte. Quest'ultimo elemento si è rivelato decisivo: un'ipotesi iniziale plausibile ma errata (`-sS`) è stata corretta proprio grazie al confronto tra la teoria e l'evidenza sperimentale, un passaggio che ha rafforzato la solidità delle conclusioni finali rispetto a un'attribuzione basata solo su indizi superficiali.
Allo stesso modo, l'identificazione del bersaglio non si è fermata al semplice hostname annunciato in chiaro, ma è stata corroborata incrociando più indicatori indipendenti (TTL, presenza di servizi Unix-specifici, assenza dei servizi tipici di Windows), a dimostrazione di come l'*OS fingerprinting passivo* permetta di trarre conclusioni affidabili anche senza accesso diretto al sistema osservato.
Dal punto di vista dell'impatto pratico, l'esercizio evidenzia un principio applicabile a contesti reali di monitoraggio e difesa della rete: una scansione di porte, per quanto banale possa sembrare, è spesso la prima fase osservabile di una catena di attacco più ampia, e la sua identificazione tempestiva rappresenta una finestra di opportunità per intervenire prima che l'attaccante raggiunga la fase di sfruttamento vero e proprio. Le porte aperte individuate su Metasploitable2 corrispondono infatti a servizi storicamente affetti da vulnerabilità critiche, e la loro scoperta da parte di un attore esterno costituirebbe, in un contesto reale, un campanello d'allarme da non sottovalutare.
Per il lavoro futuro, questa analisi suggerisce alcune direzioni di approfondimento: la traduzione degli indicatori individuati (in particolare la combinazione "molte porte diverse, stesso IP sorgente, finestra temporale ristretta") in regole automatiche per un IDS/IPS di rete; l'estensione del metodo di verifica sperimentale ad altre tecniche di scansione (es. `-sU` per UDP, o scansioni con evasione tramite decoy/frammentazione) per costruirne un catalogo di fingerprint di riferimento; e, più in generale, l'adozione sistematica dell'approccio "ipotesi → riproduzione → conferma" impiegato in questo report come buona pratica per qualunque attività di analisi forense di rete, in cui un'attribuzione non validata sperimentalmente resta, nella migliore delle ipotesi, solo un'ipotesi.